ESOF 322

Software Engineering

LECTURE: 2

J. L. Pach

OUTLINE:

- History of GIT
- GIT
- Review of Navigating the Command Line and Bash
- Git Settings
- etc

HISTORY

of GIT

1. CVS – CONCURRENT VERSIONS SYSTEM (C. 1990–2008)

One of the first widely used version control systems.

Characteristics:

- Managed versions of individual files, not entire projects.
- Centralized model – a single main repository.

Limitations:

- Poor branching support.
- No strong data integrity.
- Slow operations.
- Used in early open-source projects, including Linux in its early stages.

2. BITKEEPER (2000–2018)

A **commercial**, distributed version control system known for speed.
From 2002, it was **free for the Linux community** under a special license.

Why it mattered:

- Linus Torvalds used it to manage the Linux kernel codebase.

Crisis in 2005:

- Andrew Tridgell created a tool called SourcePuller by reverse-engineering BitKeeper's protocol.
- BitMover (Larry McVoy's company) considered this a license violation and revoked free access.

2. BITKEEPER (2000–2018)

What happened next:

- In 2016, BitKeeper was open-sourced under the Apache 2.0 license.
- Last release: 2018.
- Today, it’s practically abandoned—Git completely replaced it.

3. THE BIRTH OF GIT (2005)

After losing BitKeeper, Linus Torvalds set requirements for a new tool:

- **Free and open source.**
- **Distributed** (every user has the full history).
- **Extremely fast** (apply a patch in <3 seconds).
- **Resilient to corruption.**

Timeline:

- April 3, 2005 – development begins.
- June 2005 – first release.

3. THE BIRTH OF GIT (2005)

The name ***Git***:

- *Global Information Tracker* (when it works well).
- *Goddamn Idiotic Truckload of* **** (when it doesn't).

  Officially: *the stupid content tracker.*

GIT - GLOBAL INFORMATION TRACKER

- Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.
- Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

Docs: https://git-scm.com/doc

WHAT IS GIT? (SIMPLE DEFINITION FOR BEGINNERS)

Git is a version control system. Its main job is to track every change in your code and allow you to go back to previous versions if needed. It also lets you compare changes between versions.

But **Git** does more than that:

1. It allows multiple developers to work on the same project at the same time by creating alternative versions of the code, called branches.
2. These branches can later be merged back together. If two people changed the same part of the code, Git will show a conflict that needs to be resolved.
3. This makes Git perfect for collaborative work and for projects that evolve quickly.

GIT IN AGILE & PLAN-DRIVEN

- In Agile development, where code changes often and quality can drop over time, Git helps by making refactoring and merging much easier.
- In more rigid, plan-driven approaches, branching and merging might be less frequent, but Git is still useful for tracking history and avoiding mistakes.


SM

THANK

You